# Emergency Smart Flood Alerting System for Safer Driving in Saudi Arabia

Prepared for:
Dr. Arwa Basabreen

Shatha Alsaidy - 2009053

Danah Alghanmi - 2105578

Wasan Almutaani - 2115109

Wed Aljahdali - 2105502

# Table of Content

## 1 Introduction

This Report presents a comparative study of various approaches to flood detection using computer vision. We explore both classical feature extraction techniques, such as HOG (Histogram of Oriented Gradients) and SIFT (Scale-Invariant Feature Transform), as well as modern deep learning methods, particularly Convolutional Neural Networks (CNN). Additionally, transfer learning with pre-trained models is employed to enhance detection accuracy and efficiency. The objective of this work is to evaluate the performance of different models in identifying flooded areas and to propose an effective solution that can serve as an early warning mechanism, ultimately reducing the risk and impact of flood-related incidents.

## 2 Problem Statement

In Saudi Arabia, Flooding is a growing concern as occasional heavy rainfall leads to significant damage to infrastructure, property, and human lives. Despite the increasing frequency of rainstorms, the country lacks comprehensive and reliable flood detection and monitoring systems to respond effectively in real-time. Traditional methods for flood detection are often limited in accuracy and scalability, and there is a critical need for advanced solutions that can provide timely, precise, and actionable insights. The absence of automated flood detection systems hampers disaster response and complicates efforts to minimize the impact of floods. This paper addresses the need for more robust flood detection systems by exploring the potential of computer vision-based models for flood detection in Saudi Arabia.

## 3 Objectives

Our project aims to create a system that uses computer vision to detect floods in real-time and send alerts, improving road safety in Saudi Arabia. This system uses image classification, object detection, and segmentation to check and analyze road conditions, alerting drivers during bad weather. This use of computer vision helps improve public safety and shows how important it can be in responding to natural disasters, helping to build smarter infrastructure.

## 4 Dataset Analysis

Datasets are essential for developing, training, and evaluating machine learning models. Their quality and structure directly impact model performance in tasks like classification, detection, and segmentation. This section focuses on publicly available flood-related datasets we have used for building our system that detects and manages floods.

**1. Binary classification Flood/NoFlood :**
The Binary Classification Flood/No-Flood Dataset, sourced from Kaggle, is designed to train and evaluate models for distinguishing between flooded and non-flooded scenes. It consists of approximately 1,000 images evenly divided into 'Flood' and 'No Flood' classes, ensuring a balanced dataset for training, and testing. Key characteristics of the dataset include a wide variety of image resolutions and diverse environmental conditions, such as different lighting, weather patterns, and geographical settings. This diversity ensures that models trained on the dataset can generalize well to real-world flood detection scenarios. For classification, images are labeled by class (0 for No Flood and 1 for Flood). However, challenges do exist; despite the overall balance, certain subsets might experience class imbalance, potentially affecting model sensitivity. Additionally, while the total sample size is adequate for classification, ensuring high-quality and consistent labels is crucial, as any inaccuracies can degrade model performance. [2]

**Visualizations:**


Figure 1: Random Sample of Flood and Non-Flood Images with Labels (0: Non-Flood, 1: Flood)

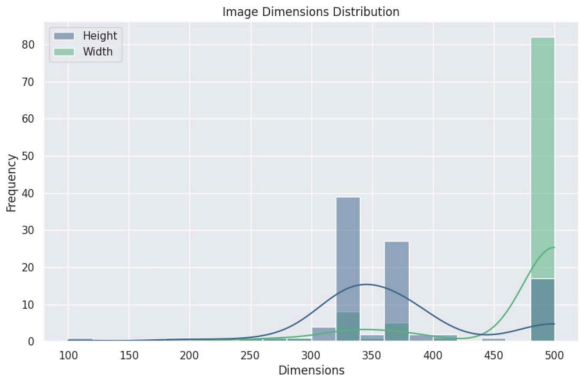


Figure 2: Distribution of Image Dimensions (Height and Width) in the Dataset

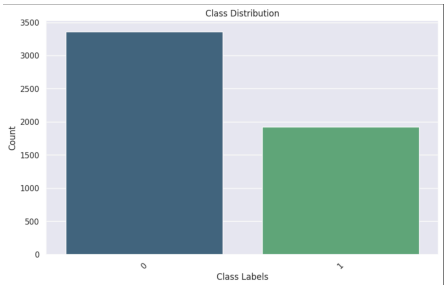


Figure 3: Distribution of Flood (Label: 1) and Non-Flood (Label: 0) Images in the Dataset

**2. flood-dataset-4gb**

The *Flood Dataset (4GB)*, sourced from Kaggle, is specifically designed to support the training and testing of machine learning models for binary classification and image recognition tasks related to flooding. The dataset is organized into three subsets: `train`, `test`, and `valid`, ensuring a clear division for model development, validation, and evaluation. Each subset contains images labeled as either ”Flooding” or ”Normal,” making it ideal for binary classification tasks. The training set includes 2,883 images of flooded scenes and 655 images of normal conditions, providing a substantial dataset for model learning. The validation set contains 42 flooding images and 39 normal images, allowing for fine-tuning and hyperparameter adjustments. The testing set comprises 441 flooding images and 29 normal images, enabling robust evaluation of model performance [3].

**Visualizations:**

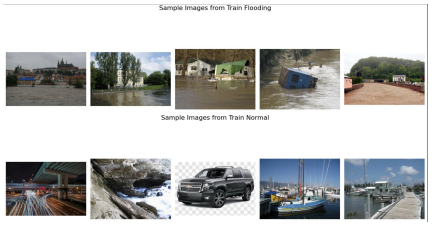


Figure 4: Sample images from the Test dataset: Flooding (top) and Normal (bottom).

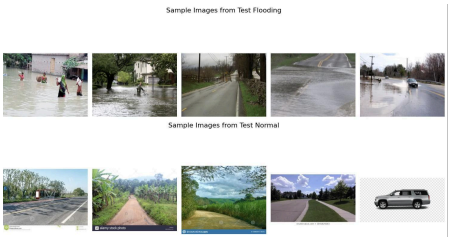


Figure 5: Sample images from the Test dataset: Flooding (top) and Normal (bottom).

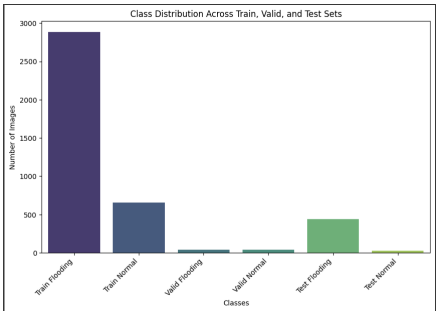


Figure 6: Class distribution of Flooding and No mal images across Train, Validation, and Test sets.

### 3. Flood Area Segmentation

The *Flood Area Segmentation Dataset*, sourced from Kaggle, is designed for training and evaluating models to segment water regions in flood-affected areas. It contains 290 images of flood-hit scenes along with 290 annotated mask images, all manually labeled using *Label Studio*. The dataset is structured into:

- **Image Folder**: Contains all flood scene images.
- **Mask Folder**: Includes corresponding segmentation masks.
- **Metadata File (**`metadata.csv|)`: Maps each image to its respective mask for seamless data management.

The total size of the dataset is **581 files**, making it compact yet sufficient for developing segmentation models. The dataset is publicly available under the CC0 license, enabling unrestricted use for research in disaster management, flood monitoring, and emergency planning [4].

**Visualizations:**

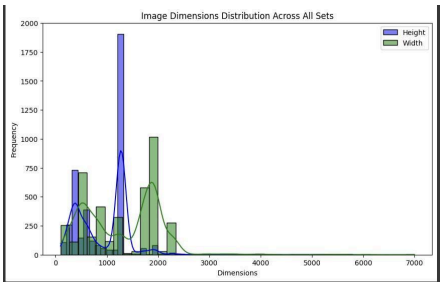


Figure 7: Image Dimensions Distribution Across All Sets (Height and Width).


Figure 8: Flood image (left) and its flood region mask (right)

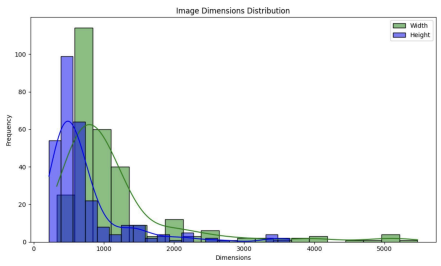


Figure 9: Image Dimensions Distribution Across the Dataset (Height and Width)

## 4. Flood Area for Segmentation, and Detection Dataset

The Flood dataset available on Roboflow is meticulously curated for advanced computer vision tasks, specifically segmentation and detection, aimed at identifying and localizing flood-affected areas within various environments. The dataset is split into a training set of 4,782 images (92%) and a validation set of 396 images (8%), with no separate test set provided. It is preprocessed by resizing images to 640x640pixels and applying auto-orientation, alongside augmentations such as horizontal and vertical flips, cropping (0% minimum zoom and 20% maximum zoom), rotations between -15° and +15°, and mosaic transformations. The annotations are provided in two formats: JSON COCO for segmentation tasks, which supports pixel-level annotations, and YOLOv8 for detection tasks, offering bounding box labels for localized flood areas. The dataset features diverse images captured under various environmental conditions, including different lighting, weather, and geo- graphical settings, making it robust for training models to generalize effectively. However, potential challenges include the absence of a test set for independent evaluation and any inherent class imbalance within subsets, which could affect the model's sensitivity to minority classes. Additionally, the high resolution and detailed annotations demand substantial computational resources for processing [1].

**Visualizations:**

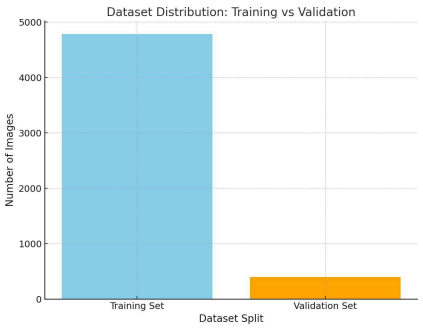


Figure 10: Dataset Distribution: Training Vs Vali dation


Figure 11: Examples Images From Dataset

## 5. Flooded Objects Dataset

The “Flooded Objects” dataset, sourced from Roboflow, contains 2886 annotated flood images, each of which is 640x640 pixels in size and in grayscale. The dataset includes two detection classes: flooded car and flooded human. It is divided into 2472 images for training, 175 images for validation, and 239 images for testing [5].

**Visualizations:**

Figure 12: Sample images from the training dataset.

Figure 13: Sample images from the validation dataset.

Figure 14: Sample images from the test dataset.

### 6. Water Levels Dataset

The ’Water Level’ dataset, sourced from Roboflow, consists of 4,370 annotated color images, each with a resolution of 640x640 pixels, designed for training computer vision-based flood and water level monitoring systems. The dataset includes 13 classes: one for flood detection and 12 representing various water levels. It is divided into 3,825 images for training, 363 for validation, and 182 for testing [6].

**Visualizations:**

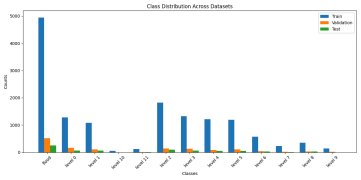


Figure 15: Class distribution of the Water Levels dataset.

Figure 16: Sample images from the training dataset.

Figure 17: Sample images from the validation dataset.

Figure 18: Sample images from the test dataset.

## 5 Preprocessing and Augmentation

In the context of flood detection using computer vision, preprocessing and data augmentation are crucial for ensuring high model accuracy and robust performance. Below are the techniques applied in this study:

### 5.1 Preprocessing Techniques

Preprocessing transforms raw image data into a suitable format for model training, ensuring consistency and enhancing learning performance. The following preprocessing steps were applied:

- **Image Resizing:** All images were resized to a fixed resolution to maintain uniform input dimensions for the models.
- **Normalization:** Pixel values were scaled to the range [0, 1] to facilitate faster convergence during training.
- **Grayscale Conversion:** For models relying on texture-based features like SIFT and HOG, images were converted to grayscale.

### 5.2 Data Augmentation Techniques

Data augmentation increases dataset diversity, reducing overfitting and improving model generalization. The following augmentation techniques were implemented:

- **Rotation:** Random rotations up to 45 to simulate different camera angles.
- **Flipping:** Horizontal and vertical flips to ad- dress spatial variability in flood scenes.

These preprocessing and augmentation techniques collectively ensured that the models were trained on a diverse and representative dataset, enhancing their ability to detect floods in various real-world conditions.

## 6 Experimental Setup

The experiments were conducted using Google Colab, using its T4 GPU for accelerated computations. The training environment included Python 3.9 and PyTorch 1.12, with the necessary libraries pre-installed in the Colab environment. The datasets were uploaded to Google Drive to ensure seamless access and integration within the notebook workflow.

In terms of training hyperparameters, they were configured as follows: the learning rate was mostly set to 0.001 with the Adam optimizer, a batch size of 32 was used for the classification models,16 for the detection models, and lastly 4 for the segmentation models due to memory limits. Training was performed for up to 10 epochs for heavy models and 50 epochs for more efficient ones. Binary cross-entropy loss was used for classification models, bounding box \& class prediction loss for the detection models, and lastly, pixel-wise loss was used with the segmentation models.

Evaluation metrics for classical classification models included accuracy, precision, and recall. For CNN models (both custom and pre-trained), the F1-score was additionally used to provide a more balanced assessment. Precision, recall and mean Average Precision (mAP@0.5) are used for the detection models, and intersection over union (IoU) for the segmentation models. Additionally, k-fold cross-validation (k=5) was employed to ensure the results' robustness, where the dataset was partitioned into five subsets, with each subset used as a validation set while training on the remaining four. The final results were obtained by averaging the metrics across all folds, ensuring consistency and reducing variability in performance evaluation.

## 7 Results and Discussion

In this section, we present and discuss the results of the experiments conducted for the flood detection tasks. The performance of various machine learning models and feature extraction techniques was analyzed for each dataset. Key observations and trends were highlighted, providing insights into the effectiveness of different methods and their potential applications.

### 7.1 Classification Task

#### 7.1.1 Classical Models

Table 1: Classical Models Classification

| Model | Feature Extraction Method | Training | | | Cross Validation Average Accuracy |
|---|---|---|---|---|---|
| | | Accuracy | Precision | Recall | |
| **Flood Dataset** | | | | | |
| SVM | HOG | 0.6265 | 0.6355 | 0.6265 | 0.6449 |
| SVM | SIFT | 0.5649 | 0.5539 | 0.5649 | 0.5591 |
| SVM | LBP | 0.7217 | 0.7230 | 0.2981 | 0.7042 |
| SVM | ORB | 0.6800 | 0.5775 | 0.2284 | 0.6629 |
| KNN | HOG | 0.6306 | 0.6078 | 0.6306 | 0.5958 |
| KNN | SIFT | 0.5886 | 0.5583 | 0.5886 | 0.5691 |
| KNN | LBP | 0.6895 | 0.5510 | 0.4819 | 0.6771 |
| KNN | ORB | 0.6068 | 0.4225 | 0.1616 | 0.6149 |
| ANN | HOG | 0.7175 | 0.7121 | 0.7175 | 0.6931 |
| ANN | SIFT | 0.5782 | 0.5619 | 0.5782 | 0.5823 |
| ANN | LBP | 0.6847 | 0.6517 | 0.6847 | 0.6804 |
| ANN | ORB | 0.6619 | 0.5231 | 0.0947 | 0.6408 |
| **Flood/NoFlood Dataset** | | | | | |
| SVM | HOG | 0.81 | 0.81 | 0.81 | 0.81 |
| SVM | SIFT | 0.73 | 0.72 | 0.73 | 0.71 |
| SVM | LBP | 0.64 | 0.59 | 0.58 | 0.6834 |
| SVM | ORB | 0.61 | 0.56 | 0.63 | 0.5758 |
| KNN | HOG | 0.78 | 0.74 | 0.76 | 0.75 |
| KNN | SIFT | 0.74 | 0.73 | 0.74 | 0.73 |
| KNN | LBP | 0.64 | 0.57 | 0.73 | 0.6688 |
| KNN | ORB | 0.52 | 0.47 | 0.50 | 0.5398 |
| ANN | HOG | 0.81 | 0.81 | 0.81 | 0.81 |
| ANN | SIFT | 0.71 | 0.70 | 0.71 | 0.7056 |
| ANN | LBP | 0.78 | 0.76 | 0.64 | 0.6595 |
| ANN | ORB | 0.60 | 0.54 | 0.55 | 0.6249 |

The results of the classical models in Table 1 demonstrate notable variations in model performance depend- ing on the feature extraction method and dataset used. For the Flood dataset, LBP stands out as the most effective feature extraction method, particularly when paired with the SVM model, which achieves the highest cross-validation accu- racy 0.7042. ANN also performs well with LBP 0.6804, indicating its adaptability to this feature set. However, the recall values for some combi- nations, such as SVM with ORB and ANN with ORB, are considerably low, highlighting chal- lenges in correctly identifying positive cases with these methods. Conversely, HOG demonstrates relatively consistent but lower performance across models, while SIFT consistently underperforms, particularly with KNN.

In the Flood/NoFlood dataset, the results im- prove significantly across all models. SVM with HOG achieves the highest accuracy 0.81, pre- cision, and recall, underscoring its effectiveness with this feature extraction method. KNN and ANN also exhibit strong performance with HOG and LBP, achieving cross-validation accuracies above 0.7. However, SIFT and ORB continue to lag in performance, with ORB producing the low- est accuracies across all models. These results in- dicate that feature extraction methods like HOG and LBP are particularly well-suited for distin- guishing between flood and non-flood conditions, while SIFT and ORB may not capture the criti- cal features necessary for high classification per- formance.

### 7.1.2 CNN Models

Table 2: CNN Models Classification

| Model | Dropout | Training | | | | Cross Validation Average Accuracy |
|---|---|---|---|---|---|---|
| | | Accuracy | Precision | Recall | F1-Score | |
| **Flood Dataset** | | | | | | |
| EfficientNet | Yes | 0.91 | 0.91 | 0.91 | 0.91 | 0.9840 |
| EfficientNet | No | 0.91 | 0.91 | 0.91 | 0.91 | 0.9848 |
| ResNet50 | Yes | 0.72 | 0.71 | 0.72 | 0.7204 | 0.4321 |
| ResNet50 | No | 0.73 | 0.73 | 0.73 | 0.7323 | 0.4321 |
| Custom Model | Yes | 0.79 | 0.80 | 0.79 | 0.79 | 0.8420 |
| MobileNet | Yes | 0.98 | 0.82 | 0.84 | 0.82 | 0.9870 |
| **Flood/NoFlood Dataset** | | | | | | |
| Custom Model 1 | Yes | 0.78 | 0.74 | 0.78 | 0.76 | 0.9600 |
| Custom Model 1 | No | 0.75 | 0.67 | 0.84 | 0.75 | 0.9400 |
| Custom Model 2 | Yes | 0.80 | 0.80 | 0.80 | 0.80 | 0.8100 |
| Custom Model 2 | No | 0.84 | 0.76 | 0.69 | 0.73 | 0.8300 |
| Inception | No | 0.95 | 0.95 | 0.95 | 0.95 | 0.9300 |
| ResNet50 | No | 0.60 | 0.71 | 0.51 | 0.59 | 0.59 |

The results for the CNN models presented in Tabel 2 indicate strong performance on the Flood dataset, with EfficientNet achieving the high- est cross-validation accuracy (0.9848) regardless of whether dropout is applied. MobileNet also performs exceptionally well, achieving a cross- validation accuracy of 0.9870, although its F1- score (0.82) is slightly lower compared to EfficientNet. ResNet50, on the other hand, struggles with this dataset, showing a significantly lower cross-validation accuracy of 0.4321. The custom CNN model performs moderately, with a cross- validation accuracy of 0.7980, suggesting its po- tential for improvement.

For the Flood/NoFlood dataset, Inception achieves the best performance, with an impres- sive cross-validation accuracy of 0.9300, followed by Custom Model 1 (0.9600 with dropout). Cus- tom Model 2 performs well (0.8100 with dropout) but falls behind Inception and Custom Model 1. ResNet50 continues to underperform in this dataset as well, with the lowest cross-validation accuracy of 0.59. These results highlight that pre- trained architectures like EfficientNet, MobileNet, and Inception are highly effective for flood detec- tion tasks, while models like ResNet50 may re- quire further fine-tuning or are less suited for these datasets. Dropout appears to have a marginal im- pact on performance in most cases, maintaining or slightly reducing accuracy without major perfor- mance losses.

## 7.2 Detection Task

Table 3: Detection Models Results

| Model | Dataset | Training | | | Cross Validation | | |
|---|---|---|---|---|---|---|---|
| | | Precision | Recall | MAP50 | Precision | Recall | MAP50 |
| YoloV5 | Flooded Objects | 0.73 | 0.77 | 0.76 | 0.69 | 0.67 | 0.69 |
| YoloV8 | Flood Area | 0.73 | 0.56 | 0.62 | 0.88 | 0.84 | 0.92 |
| YoloV11 | Flooded Objects | 0.68 | 0.81 | 0.78 | 0.79 | 0.74 | 0.81 |

The results for the detection models presented in Table 3 highlight differences in performance across various datasets. YoloV5 achieves moderate training results with a precision of 0.73, re- call of 0.77, and MAP50 of 0.76, and maintains reasonable cross-validation metrics (precision of 0.69, recall of 0.67, and MAP50 of 0.69). This suggests that YoloV5 performs consistently but has limitations in generalizing to unseen data.

YoloV8 demonstrates strong cross-validation performance, achieving the highest MAP50 (0.92), precision (0.88), and recall (0.84) among the models. However, its training metrics, particu- larly recall (0.56) and MAP50 (0.62), are lower compared to YoloV5 and YoloV11. This dis- discrepancy indicates that YoloV8 likely focuses on higher precision during cross-validation but strug- gles to capture all relevant objects during training.

YoloV11 achieves the best training perfor- mance, with the highest MAP50 (0.78) and re- call (0.81), demonstrating its ability to detect a wide range of objects effectively during training. Its cross-validation metrics, while slightly lower than YoloV8, remain competitive, with a MAP50 of 0.81 and a balanced precision (0.79) and re- call (0.74). This makes YoloV11 a reliable model for scenarios requiring both strong training perfor- mance and robust cross-validation results. Over- all, the results suggest that YoloV8 is optimal for high-precision detection tasks, while YoloV11 provides a balanced approach for broader object detection tasks.

### 7.3 Segmentation Task

Table 4: Segmentation Models Results

| Model | Dataset | Training | | | | | Cross Validation |
|---|---|---|---|---|---|---|---|
| | | Pixel Accuracy | IoU | Precision | Recall | F1-score | IoU |
| Custom Model#1 (Unet-based) | Water Levels | 0.74 | 0.12 | 0.22 | 0.16 | 0.17 | 0.12 |
| Custom Model#2 (Unet-based) | Flood Area Segmentation | 0.84 | 0.55 | 0.98 | 0.98 | 0.98 | 0.73 |
| YoloV8#1 | Flood Area | 0.74 | 0.74 | 0.99 | 0.74 | 0.85 | 0.80 |
| YoloV8#2 | Water Levels | 0.97 | 0.95 | 0.81 | 0.65 | 0.72 | 0.88 |

The segmentation results presented in Table 4 highlight the performance of various models across different datasets. The Custom Model#1 (Unet-based) demonstrates the weakest perfor- mance on the Water Levels dataset, with a low IoU of 0.12 and an F1-score of 0.17, suggesting lim- ited segmentation capabilities for this task. On the other hand, Custom Model#2 (Unet-based) per- forms significantly better on iiii#1, achieving a Pixel Accuracy of 0.84, IoU of 0.55, and an F1- score of 0.98, showcasing its suitability for this particular dataset.

The YoloV8 models exhibit superior perfor- mance across all datasets. YoloV8#1 achieves strong results on the iiii#2 dataset, with an IoU of 0.74 and an F1-score of 0.85, demonstrating its effectiveness in handling this dataset. Notably, YoloV8#2 delivers the highest scores overall on the Water Levels dataset, with a Pixel Accuracy of 0.97, IoU of 0.95, and an F1-score of 0.72, em- phasizing its robustness and generalization capa- bilities. These results underscore the effectiveness of YoloV8 models compared to the custom Unet- based models, while also highlighting how dataset characteristics significantly influence model per- formance. Further analysis will explore the rela- tionship between model architecture and dataset complexity.

## 8 Conclusion

In conclusion, this paper demonstrated that certain models significantly excel in flood detection tasks. When it comes to feature extraction, methods such as LBP and HOG achieved high accuracy, particularly when combined with SVM and Artificial Neural Networks (ANN). Additionally, pre-trained architectures like EfficientNet, MobileNet, and Inception proved to be highly effective for detection tasks using CNN models. Notably, the results indicated that YOLOv8 performs exceptionally well in both segmentation and image detection tasks, underscoring its utility in flood alerting systems.

## 9 References


1. K. Pham, "Binary Classification: Flood/No-Flood Dataset," Accessed: Dec. 14, 2024, 2021. [Online]. Available: https://www.kaggle.com/datasets/khuongpd/binary-classification-floodnoflood?select=devset_images_gt.csv.
2. "Flood Dataset (4GB)," Available: https://www.kaggle.com/datasets/phamtuananhk16hl/flood-dataset-4gb/code. [Accessed: Dec. 14, 2024].
3. F. Karim, "Flood Area Segmentation Dataset," Accessed: Dec. 14, 2024, 2021. [Online]. Available: https://www.kaggle.com/datasets/faizalkarim/flood-area-segmentation.
4. L. N. Hoang, "Flood Dataset," Open Source Dataset, Accessed: Dec. 14, 2024. Available: https://universe.roboflow.com/long-nguyen-hoang-9ecmq/flood-4oe1x.
5. S. T. Dhamad, "Flooded Objects Dataset," Accessed: Dec. 14, 2024. [Online]. Available: https://universe.roboflow.com/sahar-turki-dhamad/flooded-objects.
6. Q. Ali, "Water Level Sindh Dataset," Accessed: Dec. 14, 2024. [Online]. Available: https://universe.roboflow.com/qurban-ali-qvlhx/water-level-sindh.